

Chapter 24 - IE64

IE64 is the native processor of the Intuition Engine. Every other CPU in Part IV is in some sense a guest: a smaller instruction set carried along for compatibility with older software. IE64 is where the system runs at full speed and where BASIC itself lives.

It is a clean 64-bit RISC load-store architecture with fixed 8-byte instructions, 32 general-purpose registers, and no flags. Comparisons fold into the branches that consume them.

24.1 Register file

IE64 has 32 general-purpose registers, each 64 bits wide:

Register	Role
R0	Hardwired zero. Writes are silently ignored; reads always return 0.
R1–R7	Caller-saved scratch. Free for any subroutine to clobber.
R8–R15	Argument and return registers (by convention). R8 is the first argument and the primary return value.
R16–R30	Callee-saved (by convention). EhBASIC reserves R16, R17, R26, R27 for BASIC-wide use.
R31	Stack pointer (SP). Decrements before push, increments after pop.

R0 = zero is a hardware fact; the rest are conventions. Nothing in silicon stops a program from using R31 for general data and keeping the stack elsewhere, but every interrupt handler and every library routine assumes the convention.

The FPU adds 16 separate registers, F0–F15, each holding one IEEE 754 single-precision (FP32) value in its low 32 bits. Double-precision instructions operate on pairs of F registers.

Three architecturally visible state words sit beside the GPRs:

- PC - the program counter, 64 bits.
- FPSR / FPCR - floating-point status and control. See §24.5.
- A bank of 16 control registers CR0–CR15. See §24.7.

24.2 Instruction encoding

Every instruction is exactly 8 bytes long, little-endian, with this layout:

byte 0	: opcode	(8 bits)
byte 1	: Rd[4:0] size[1:0] X	(5 + 2 + 1)
byte 2	: Rs[4:0] reserved	(5 + 3)
byte 3	: Rt[4:0] reserved	(5 + 3)
bytes 4-7	: imm32	(32 bits)

Size codes are B = 0 (8-bit), W = 1 (16-bit), L = 2 (32-bit), Q = 3 (64-bit). An instruction that does not need a size code ignores those bits.

Fixed length makes the decoder simple and gives the disassembler a predictable cadence: each address 0x1000, 0x1008, 0x1010, ... holds one full instruction.

Instructions are always aligned to 8 bytes. A branch target that is not so aligned faults.

24.3 Addressing modes

The load-store architecture means only LOAD and STORE (and their floating-point and atomic siblings) touch memory; every other operation works register-to-register.

LOAD/STORE accept:

- **Register indirect:** `LOAD.x Rd, (Rs)` reads x bytes from the address in Rs .
- **Register-plus-displacement:** `LOAD.x Rd, imm(Rs)` reads x bytes from $Rs + \text{imm32}$. The displacement is signed.
- **Absolute:** `LOAD.x Rd, (imm)` reads x bytes from the absolute address imm32 .

The size code `.B`, `.W`, `.L`, `.Q` selects the access width. `LOAD.B` zero-extends a byte into a 64-bit register; sign-extend explicitly with the `SEXT` instruction if you want a negative byte to sign-extend.

LEA (Load Effective Address) computes the address that a LOAD/STORE would have used, without actually touching memory: `LEA Rd, imm(Rs)` puts $Rs + \text{imm32}$ in Rd .

The assembler's `LA Rd, label` pseudo-instruction expands to a small sequence (typically `MOVE` followed by `MOVT`) that loads a full 64-bit absolute address into Rd .

24.4 Integer instructions

24.4.1 Data movement

Mnemonic	Effect
<code>MOVE Rd, Rs</code>	Copy Rs into Rd
<code>MOVE.Q Rd, #imm</code>	Move sign-extended 32-bit immediate into Rd
<code>MOVT Rd, #imm</code>	Move into upper 32 bits of Rd (paired with <code>MOVE</code> for full 64-bit constants)
<code>MOVEQ Rd, Rs</code>	Sign-extend low 32 bits of Rs into 64-bit Rd
<code>LEA Rd, imm(Rs)</code>	Address calculation, no memory access

24.4.2 Memory

Mnemonic	Effect
<code>LOAD.x Rd, ...</code>	Load x bytes from memory into Rd (zero-extended)
<code>STORE.x Rd, ...</code>	Store low x bytes of Rd to memory

24.4.3 Arithmetic

`ADD`, `SUB`, `MULU`, `MULS`, `DIVU`, `DIVS`, `MOD`, `MODS`, `NEG`, `MULHU`, `MULHS`. All three-operand register-register, all 64-bit. The two `MULH` variants return the high 64 bits of a 64×64 product.

Division by zero raises a fault. Programs that anticipate untrusted divisors should check the divisor against zero first.

24.4.4 Logic and shifts

`AND`, `OR`, `EOR (XOR)`, `NOT`. Shifts: `LSL`, `LSR`, `ASR`, `ROL`, `ROR`. Bit utilities: `CLZ` (count leading zeros), `CTZ` (count trailing zeros), `POPCNT`, `BSWAP` (32-bit byte swap).

SEXT sign-extends a sub-64-bit value to the full register width; size code selects the source width.

24.4.5 Branches

IE64 has no flags register. Each branch instruction is a compare-and-branch in one step:

Mnemonic	Branches if (signed unless noted)
BRA	unconditional
BEQ Rs, Rt, label	Rs == Rt
BNE Rs, Rt, label	Rs != Rt
BLT Rs, Rt, label	Rs < Rt
BGE Rs, Rt, label	Rs >= Rt
BGT Rs, Rt, label	Rs > Rt
BLE Rs, Rt, label	Rs <= Rt
BHI Rs, Rt, label	Rs > Rt (unsigned)
BLS Rs, Rt, label	Rs <= Rt (unsigned)
JMP Rs	unconditional register-indirect jump

The assembler provides BEQZ Rs, label as shorthand for BEQ Rs, R0, label; similarly BNEZ and the others against R0.

24.4.6 Subroutines

Mnemonic	Effect
JSR label	Push return PC to stack (R31), jump to label
JSR Rs	Push return PC, jump to Rs (register-indirect)
RTS	Pop return PC from stack, jump there
PUSH Rs	R31 -= 8; [R31] = Rs
POP Rd	Rd = [R31]; R31 += 8

The stack grows downward. By convention R31 starts at the top of the per-program stack region (BASIC uses STACK_TOP = 0x9F000).

24.5 Floating-point unit

The FPU implements IEEE 754 single-precision (FP32) on F registers, plus a double-precision (FP64) extension that operates on register pairs. Operations are register-to-register. Memory operands transit through FLOAD/FSTORE for FP32 and DLOAD/DSTORE for FP64.

Mnemonic	Operation
FMOV	Copy F register
FLOAD	Memory -> F register (32 bits)
FSTORE	F register -> memory (32 bits)

Mnemonic	Operation
FADD/FSUB/FMUL/FDIV/FMOD	Basic arithmetic
FABS/FNEG/FSQRT	Unary
FINT	Round to integer (mode set in FPCR)
FCMP	Compare two FP values, write integer - 1/0/1 to a GPR
FCVTIF	Integer -> float
FCVTFI	Float -> integer (truncate toward zero)
FMOVI	Bitwise GPR -> F (no conversion)
FMOV0	Bitwise F -> GPR
FSIN/FCOS/FTAN/FATAN	Trig
FLOG/FEXP/FP0W	Log / exp / power
FMOVECR	Load a ROM constant by index
FMOVSR/FMOVCR	Read FPSR / FPCR
FMOVSC/FMOVCC	Write FPSR / FPCR

Double-precision (DMOV, DLOAD, DSTORE, DADD, ... DCMP, DCVTIF, DCVTFI) operate on FP register pairs that hold a single 64-bit value across two F slots. FCVTSD and FCVTDS convert between the two formats.

FPCR (FP control) holds the rounding mode (0=nearest, 1= toward zero, 2=down, 3=up). EhBASIC sets it to 1 at startup because INT(x) is defined to truncate toward zero.

FPSR (FP status) latches exception flags: inexact, overflow, underflow, invalid, divide-by-zero. Read with FMOVSR. The FPU does not trap on these flags; software polls FPSR if it cares.

24.6 Atomic memory operations

Six atomic read-modify-write instructions, each operating on a naturally-aligned 64-bit word:

Mnemonic	Operation
CAS Rd, Rt, (Rs)	If [Rs] == Rd then [Rs] = Rt; Rd = old
XCHG Rd, Rt, (Rs)	old = [Rs]; [Rs] = Rt; Rd = old
FAA Rd, Rt, (Rs)	old = [Rs]; [Rs] = old + Rt; Rd = old
FAND Rd, Rt, (Rs)	old = [Rs]; [Rs] = old & Rt; Rd = old
FOR Rd, Rt, (Rs)	old = [Rs]; [Rs] = old \ Rt; Rd = old
FXOR Rd, Rt, (Rs)	old = [Rs]; [Rs] = old ^ Rt; Rd = old

A misaligned address (not divisible by 8) raises FAULT_MISALIGNED. All six are sequentially consistent on the memory subsystem.

24.7 Privilege and the MMU

IE64 has two privilege levels: supervisor and user. A program that boots from the image executor runs in supervisor mode. EhBASIC runs in supervisor mode and uses the full address space directly.

A program that wants user-mode isolation enables the MMU through the CR_MMU_CTRL control register.

24.7.1 Control registers

MTCR (Move To Control Register) and MFCR (Move From) access the bank:

CR index	Name	Purpose
0	PTBR	Page-table base register (physical)
1	FAULT_ADDR	VA that caused the last fault
2	FAULT_CAUSE	Fault cause code
3	FAULT_PC	PC saved at trap entry
4	TRAP_VEC	Trap handler vector
5	MMU_CTRL	MMU enable + protection flags
6	TP	Thread pointer (user-readable)
7	INTR_VEC	Timer-interrupt vector
8	KSP	Supervisor stack pointer
9	TIMER_PERIOD	Timer reload value
10	TIMER_COUNT	Current timer countdown
11	TIMER_CTRL	b0 enable, b1 IRQ enable
12	USP	Saved user stack pointer
13	PREV_MODE	Previous privilege mode
14	SAVED_SUA	Saved SUA latch
15	RAM_SIZE_BYTES	Live read of active visible RAM

CR_RAM_SIZE_BYTES is read-only; writing it raises a fault.

24.7.2 The MMU

The MMU walks a sparse 6-level radix page table. The top level indexes 7 bits of the virtual page number (128 entries); the five lower levels index 9 bits each (512 entries each), so the full VPN is 52 bits. Page size is 4 kilobytes.

Each page-table entry is 64 bits: the low 7 bits hold flags (P, R, W, X, U, A, D); bits 12-63 hold the physical page number. A missing PTE raises FAULT_NOT_PRESENT. A read of a non-readable page raises FAULT_READ_DENIED, and so on.

MMU_CTRL controls translation:

Bit	Name	Effect
0	ENABLE	Translation active
1	SUPER	Read-only: current mode is supervisor

Bit	Name	Effect
2	SKEF	Fault on supervisor instruction fetch from a user page
3	SKAC	Fault on supervisor data access to a user page (outside SUA region)
4	SUA	Supervisor-user-access latch (set by SUAEN, cleared by SUADIS)

SKEF/SKAC together give the equivalent of SMEP/SMAP: a supervisor that has set them cannot accidentally execute or read user-controlled memory.

24.7.3 Traps

The SYSCALL instruction is the user-to-supervisor doorway. It saves PC in FAULT_PC, sets FAULT_CAUSE to FAULT_SYSCALL, swaps to the supervisor stack (KSP), and jumps to TRAP_VEC. The imm32 field of the instruction is the syscall number, available to the handler.

ERET (Exception Return) restores PC from FAULT_PC, restores the previous mode from PREV_MODE, and swaps the user stack back in. Chapter 30 details the full trap sequence including how nested traps must save SAVED_SUA.

TLBFLUSH invalidates the whole address-translation cache. TLBINVAL Rs invalidates the single entry for the VPN in Rs.

SMODE Rd reports the current mode in Rd (0 user, 1 supervisor). Unlike most privilege-related instructions, SMODE is available to user code.

24.8 Calling convention

The convention used by EhBASIC and by the assembler's macro library is:

- Up to 8 integer arguments in R8-R15.
- Return value in R8.
- R1-R7 caller-saved.
- R16-R30 callee-saved.
- Stack R31, grows downward, aligned to 8 bytes.
- Floating-point arguments in F0-F7, return in F0.

Subroutines either preserve R16-R30 or push them on entry and pop on return.

24.9 Reset and boot

On reset the CPU starts at 0x0000, the VECTOR_RESET slot. The ordinary convention is for the reset vector to do nothing more than jump to PROG_START = 0x1000, which is where the image executor (Chapter 31) places loaded programs.

VECTOR_IRQ = 0x0004 is the interrupt vector when the MMU is disabled. With the MMU enabled, INTR_VEC takes over.

The initial stack pointer for BASIC is STACK_TOP = 0x9F000. The LA R31, STACK_TOP instruction is one of the first things a freshly loaded program executes.

24.10 A small example

A complete IE64 program that prints "HI" to the terminal:

```

org 0x1000

la      r31, 0x9F000      ; set up stack
la      r5, 0x000F0700    ; TERM_OUT

move.q  r1, #'H'
store.l r1, (r5)
move.q  r1, #'I'
store.l r1, (r5)
move.q  r1, #13
store.l r1, (r5)

la      r5, 0x000F07F0    ; TERM_SENTINEL
move.l  r1, #0xDEAD
store.l r1, (r5)          ; halts the CPU

.halt:
bra     .halt

```

The LA pseudo-instruction expands to MOVE + MOVNT to load the full 64-bit address. The final BRA is unreachable but acts as the conventional landing pad for a halted program.

24.11 What comes next

Chapter 25 covers IE32, the 32-bit predecessor of IE64. IE32 has fewer registers, smaller addresses, and a simpler MMU model, but otherwise looks much the same to a programmer who has already met IE64.